

HOOP – Human Object Orientated Programming Language

HOOP Documentación

Kendall Fonseca Hidalgo y Alejandro León Marin.

1. Planteamiento del problema y solución adoptada

1.1 Objetivo: ¿Por qué crear HOOP? ¿Qué problemas resuelve?

HOOP se fundamenta en resolver la dificultad de aprendizaje de lenguajes sintácticamente complejos, es un lenguaje de programación orientado a objetos que hemos diseñado con una sintaxis inspirada en el inglés natural, manteniendo conceptos accesibles para hispanohablantes, además de resolver la ambigüedad de palabras que abundan dentro de estos lenguajes; esta falta de novedad en los términos utilizados va de la mano con la falta de legibilidad dentro de su semántica. Nuestro lenguaje se propone manejar un lenguaje entendible para humanos, con semántica clara y sintaxis tipo pseudocódigo estructurado. Nuestro intérprete de HOOP sigue una arquitectura clásica de compilación en cuatro fases bien definidas, donde cada fase tiene una responsabilidad específica y se ejecuta de manera secuencial. Esta separación de responsabilidades no solo facilita el mantenimiento y la extensión del código, sino que también permite detectar errores de manera más precisa en cada etapa del proceso.

2. Descripción del lenguaje

2.1 HOOP adopta una mezcla de los paradigmas:

- Imperativo, al permitir definir instrucciones secuenciales y estructuras de control explícitas.
- Orientado a objetos, al utilizar *mold* para clases, *action* para métodos, *create* para instanciación, y *self* para referencia interna
- Posee un **tipado híbrido**, permitiendo al usuario decidir cuando sea explícito.

2.2 Palabras reservadas

2.2.1 Estructuras de definición

<i>mold</i>	clase (class)
<i>bond</i>	interfaz (interface)
<i>listing</i>	enumeración (enum)
<i>layout</i>	estructura (struct)
<i>unit</i>	paquete/módulo (package)

2.2.1 Control de flujo

when	Condicional if
otherwise	Condicional else
cycle	Bucle for
repeat	Bucle while
halt	Terminar bucle (break)
skip	Saltar iteración (continue)
answer	Retornar de función (return)
select	Estructura switch/case
case	Caso específico en select
default	Caso por defecto en select

2.2.3 Declaración

fixed	Constante (const)
data	Variable con tipado genérico/inferido (var)

2.2.4 Funciones

action	Definir función o método (fun)
--------	--------------------------------

2.2.5 Objetos / referencias

forge	Crear objeto (new)
self	Referencia a la instancia actual (this)
void	Valor nulo (null)

2.2.6 Manejo de errores

attemp	Bloque try
ensure	Bloque finally
throw	Lanzar excepción (throw)
rescue	Bloque catch

2.2.7 I/O

display	Imprimir en pantalla (print / printf)
---------	--

2.3 Palabras reservadas de tipos de datos (tipado híbrido)

2.3.1 Simples

logic	Booleano (boolean)
whole	Entero (integer)

fract	Decimal / flotante (float / double)
text	Cadena de texto (string)
char	Carácter (char)

2.3.2 Compuestos

grid	Matriz (matrix)
chain	Lista enlazada (linked list)

2.4 Palabras pregonadas (funciones internas/built-in)

length	Longitud de un arreglo o cadena
size	Tamaño de una colección
type	Tipo de dato de un valor
convert	Conversión de tipo
input	Entrada de datos por consola
output	Salida de datos
read	Leer archivo o recurso
write	Escribir archivo o recurso
open	Abrir recurso
close	Cerrar recurso
max	Valor máximo
min	Valor mínimo
abs	Valor absoluto
sqrt	Raíz cuadrada
pow	Potencia
random	Número aleatorio

2.5 Operadores

2.5.1 Aritméticos

plus	Suma (+)
minus	Resta (-)
times	Multiplicación (*)
divide	División (/)
mod	Módulo (%)

2.5.2 Comparación

equals	Igual (==)
notequals	Diferente (!=)
less	Menor (<)
greater	Mayor (>)
lesseq	Menor o igual (<=)
greatereq	Mayor o igual (>=)

2.5.3 Asignación

set	Asignación (=)
-----	----------------

2.5.3 Lógicos

and	Lógico Y (&&)
or	Lógico O (**)
not	Negación (!)

2.6 Símbolos de estructura y puntuación

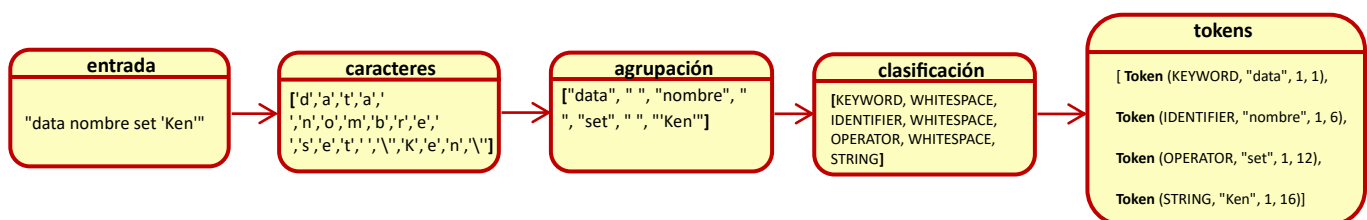
;	Fin de línea * Finaliza una instrucción
{ }	Llaves * Agrupar bloques de código
()	Paréntesis * Llamadas a funciones, agrupaciones lógicas
[]	Corchetes * Acceso a elementos en colecciones/matrices
,	Coma * Separador de elementos y de decimales
.	Punto * Acceso a miembros de un objeto
:	Dos puntos * Uso especial (por definir en el lenguaje)
+	Comentario

3. Funcionamiento de HOOP.

3.1 Etapa de análisis léxico.

Su “**LEXER**” posee un flujo de creación de Tokens, recibiendo la entrada completa, pasando por la lectura carácter por carácter, para luego agruparlos según los espacios vacíos encontrados, y clasificarlos según corresponda [KEYWORD, WHITESPACE, IDENTIFIER, WHITESPACE, OPERATOR, WHITESPACE, STRING], por último genera los tokens según corresponda con una estructura:

TOKEN (<TokenType>, <TokenContent>, <TokenColumn>, <Token Row>)



3.2 Etapa análisis sintáctico

Para HOOP se diseñó un **parser descendente recursivo** que implementa directamente la **gramática BNF oficial** del lenguaje adjunta en este documento. Este parser analiza la secuencia de tokens generados por el lexer mencionado en el paso anterior, y los utiliza para construir un **Árbol de Sintaxis Abstracta (AST)** estructurado o detectar errores sintácticos. No usa librerías externas, toda la implementación es manual utilizando **clases de nodos AST especializadas**.

Como sabemos el lexer produce una lista de tokens anteriormente vista en la etapa anterior, el parser recibe esta lista, manteniendo tres de los aspectos principales durante el parseo (posición actual en la secuencia de tokens, Token actual bajo análisis, y métodos de control para avanzar o retroceder según corresponda durante la verificación de estos tokens). Este parser trabaja mediante un descenso recursivo donde cada regla de la gramática BNF se implementa como un método individual. Cada función es responsable de reconocer una construcción específica del lenguaje. Por ejemplo, tenemos una función para reconocer declaraciones de variables, otra para reconocer funciones, otra para condicionales, y así sucesivamente.

Este parser procesa tokens secuencialmente usando tres estrategias principales:

```
match (tipo, valor): Verifica si el token actual coincide sin
avanzar.

expect(tipo, valor): Exige un token específico y avanza, o lanza
error.

Skip_newlines(): Ignora tokens de formato newlines, whitespaces,
comentarios...
```

En lugar de solo validar sintaxis, el parser construye un **AST estructurado** usando nodos especializados:

```
ProgramaNode → Contiene todas las declaraciones del programa
DeclaracionNode → Variables y constantes
FuncionNode → Definiciones de funciones
ClaseNode → Definiciones de clases (moldes)
OperacionNode → Operaciones con operadores en palabras
LiteralNode → Valores literales (números, strings, booleans)
```

Una característica única es el parsing de operadores en palabras de HOOP (set, plus, minus, greater, etc.) implementado mediante parsing de precedencia en métodos específicos:

```
parse_operacion_logica() → and, or
parse_operacion_comparacion() → equals, greater, less
parse_operacion_aritmetica() → plus, minus
parse_operacion_multiplicativa() → times, divide, mod
```

En cuanto a la detección de errores, HOOP verifica inicialmente que no haya errores léxicos para continuar con sus siguientes fases, así como verificar que no haya errores sintácticos para proceder, la metodología del parse se basa en:

1. Identifica el problema: Token esperado vs encontrado
2. Incluye contexto: Línea y posición del error
3. Lanza `ParseError`: Con mensaje descriptivo y token problemático
4. Implementa backtracking: Para distinguir entre asignaciones y expresiones

El resultado del análisis sintáctico es un árbol donde cada nodo representa una construcción del lenguaje. Por ejemplo, un nodo `'DeclaracionNode'` contiene el nombre de la variable y un nodo hijo que representa su valor inicial. Un nodo `'FuncionNode'` contiene el nombre de la función, una lista de parámetros y una lista de nodos que representan las instrucciones del cuerpo de la función. Este árbol es mucho más fácil de procesar que la lista plana de tokens porque captura explícitamente la estructura y las relaciones entre las diferentes partes del programa.

3.3 Declaraciones BNF de HOOP

<code><programa> ::= (<declaracion> <clase> <funcion> <statement>)* <EOF></code>
<code><declaracion> ::= "data" <IDENTIFIER> "set" <expresion> ";"</code>
<code><funcion> ::= "action" <IDENTIFIER> "(" <parametros>? ")" "{" <statements> "}"</code>
<code><funcion_global> ::= "action" <IDENTIFIER> "(" <parametros>? ")" "{" <statements> "}"</code>

<code><parametros> ::= <parametro> ("," <parametro>)*</code>
<code><parametros> ::= <TIPO> <IDENTIFIER> ("," <TIPO> <IDENTIFIER>)*</code>
<code><clase> ::= "mold" <IDENTIFIER> "{" <clase_cuerpo> "}"</code>
<code><clase_cuerpo> ::= (<atributo> <metodo>)*</code>
<code><statements> ::= (<statement>)*</code> <code><statement> ::= <if_statement></code> <code> <cycle_statement></code> <code> <simple_statement></code> <code> <declaracion></code>
<code><if_statement> ::= "when" <condicion> "{" <statements> "}" ["otherwise" "{" <statements> "}"]</code>
<code><cycle_statement> ::= "cycle" <IDENTIFIER> ["from" <expresion>] ["to" <expresion>] "{" <statements> "}"</code>
<code><simple_statement> ::= <expresion> ";"</code>
<code><expresion> ::= <literal></code>
<code><literal> ::= <STRING> <NUMBER> <BOOLEAN></code>
<code><operacion> ::= <IDENTIFIER> <OPERADOR> <IDENTIFIER></code>
<code><condicion> ::= <expresion> <OPERADOR_COMPARACION> <expresion></code>
<code><OPERADOR> ::= "plus" "minus" "times" "divide" "mod"</code>
<code><OPERADOR_COMPARACION> ::= "equals" "notequals" "greater" "less" "greatereq" "lesseq"</code>
<code><TIPO> ::= "logic" "whole" "fract" "text" "char" "grid" "chain"</code>
<code><KEYWORD> ::= "mold" "data" "action" "when" "otherwise" "cycle" "from" "to" "set"</code>

3.4 Etapa análisis semántico.

Después de construir el árbol de sintaxis abstracta, necesitamos verificar que el programa tenga sentido desde el punto de vista lógico y semántico. Esta es la responsabilidad del analizador semántico. Mientras que el parser verifica la estructura gramatical, el analizador semántico verifica las reglas de significado del lenguaje.

La herramienta principal del análisis semántico es la tabla de símbolos, una estructura de datos que mantiene un registro de todas las variables, funciones y clases declaradas en el programa, junto con información sobre sus tipos y ámbitos. Cuando el analizador encuentra una declaración de variable, agrega una entrada en la tabla de símbolos con el nombre de la variable y su tipo inferido. Cuando encuentra el uso de una variable, consulta la tabla para verificar que esa variable haya sido declarada previamente. También detecta reasignaciones a constantes, parámetros duplicados, profundidad de bucles (loop_depth), anidamientos máximos, matching de argumentos vs. parámetros, uso correcto de self, etc. Solo si no hay errores semánticos, el intérprete ejecuta el AST y pasa al parser.

3.4 Etapa análisis interpreter.

Una vez que el AST supera las fases léxica, sintáctica y semántica, lo procesa el intérprete. Este módulo aplica un visitante sobre cada nodo del árbol y ejecuta la semántica definida del lenguaje HOOP:

- Mantiene un ExecutionContext con scopes anidados (similar a la tabla de símbolos, pero ahora para valores). Cada contexto guarda variables, funciones y clases disponibles; cuando se llama a una función o ingresas a un ciclo, se crea un sub-contexto y se destruye al salir, conservando el encapsulamiento.
- Para cada tipo de nodo hay un método visit_*: declaraciones almacenan valores en el contexto (data registra variables, fixed registra constantes protegidas).
- _call_builtin implementa las funciones pregonadas (display, input, convert, max/min, read/write, etc.). Desde la GUI puede redirigir input() a un diálogo para que no se bloquee.
- Durante la ejecución se capturan ReturnException, LoopBreak, LoopContinue y HoopException para manejar answer, halt, skip y throw con el flujo correcto. Cualquier HoopRuntimeError se envía a la terminal o, en la GUI, a la pestaña de problemas.

En resumen, el intérprete recorre el AST, apoyándose en los contextos, da vida a la ejecución de HOOP: declara y actualiza variables, resuelve expresiones, controla el flujo y produce la salida final que ves en el terminal/GUI.

Bibliografía:

Jeffery, C. L. (2024). *Build your own programming language: A programmer's guide to designing compilers, interpreters, and DSLs for solving modern computing problems* (2.^a ed.). Packt Publishing Ltd.

Link prfe:

<https://books.google.co.cr/books?id=bDDxEAAAQBAJ&printsec=frontcover&hl=es#v=onepage&q&f=false>